

Step 4 — Forgetting, and a Privacy-Respecting Delete — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real `Store` `delete` calls), **Advanced** (what actually proves forgetting worked, the same honest-testing standard Step 1 used for persistence). Read Basic first even if `delete` already sounds simple — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Deleting every key, not just one](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Deleting every key, not just one

Basic Version

By Step 3, a user can have more than one fact saved (`preferred_name` , `stated_preference` , maybe more later). Forgetting a user means finding **every** key in their namespace and deleting each one — not just deleting one key you happen to remember the name of.

Things to use: - `store.search(namespace, query="")` (or any broad query) to list everything currently saved for that namespace. - `store.delete(namespace, key)` for each item found. - A plain, deterministic function — no LLM call needed here. Deleting is not a judgment call.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exact pieces:

- `store.search(namespace, query="all facts")` returns a list of items, each with a `.key` attribute — loop over the *result*, not over some list of keys you maintain separately, so `forget_user` doesn't silently miss a key that was added after you wrote it.
- `store.delete(namespace, key)` removes exactly one item. Call it once per item found in the search.
- In `main.py`'s input loop, check for the special command **before** it reaches the graph at all: `if text.strip().lower() == "forget_me":` should call `forget_user(user_id)` directly and print a confirmation — it should never be handed to `chat_node` as a normal chat message.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think hard about why this has to be a real delete and not a soft flag like `is_forgotten: True` sitting next to the untouched facts. A soft flag means the data still physically exists — a bug in whatever checks the flag, a future code change that forgets to check it, or simply someone with direct database access, could all still see or use it. A real `store.delete(...)` call removes the row. There's nothing left to leak later. When a user asks to be forgotten, "we stopped showing it to you" and "we actually deleted it" are very different promises, and only one of them is what `forget_me` should mean here.

The other thing worth getting right: **how do you actually prove `forget_user` worked**, versus just trusting that `store.delete(...)` didn't raise an exception? The same honest-testing standard from Step 1 applies here. A test that calls `forget_user` and then immediately checks `store.search(...)` returns empty, using the *same* Python process and the *same* `InMemoryStore` object, proves the delete call ran — but it doesn't yet prove a completely fresh run of the program would also see nothing. Since `InMemoryStore` is process-local by design (it resets on every new run anyway), the truly convincing test for this step is: seed a fact, run `forget_user`, then start a **new** graph invocation on a **brand-new thread** and confirm `recall_node` finds nothing to inject — the same "prove it as a fresh consumer, not just as the code that just ran" instinct Step 1 used for the checkpoint.

Things to try before Hint 2: - Save two different facts (`preferred_name` and `stated_preference`) for one user, call `forget_user`, then search that namespace directly and confirm zero items remain — not just the one you happened to name in your test. - After forgetting, ask "what's my name?" on a brand-new thread for that same user — confirm the answer is an honest "I don't know," not a stale cached answer from earlier in the same test run.

Difference between Basic, Intermediate, and Advanced: Basic names the idea (find everything, delete each one, no judgment call needed). Intermediate gives the real `search` -then- `delete` loop and where the `forget_me` command has to be intercepted in `main.py`. Advanced is the actual reasoning this step is built around: why a real delete matters more than a flag, and how to test forgetting the same honest way Step 1 tested remembering.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
forget_user(user_id):
    namespace = namespace_for(user_id)
    items = store.search(namespace, query for everything)
    for each item:
        store.delete(namespace, item.key)

main.py loop:
    read a line
    if line == "forget_me":
        forget_user(user_id)
        print confirmation
        continue (skip the graph entirely)
    otherwise: invoke the graph as normal
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Here's almost the whole thing — type it out yourself and adjust to your own `store.py` :

```
# store.py (adds to Step 2's get_store / namespace_for)
def forget_user(user_id: str) -> int:
    """Delete every saved fact for this user. Returns how many were deleted."""
    store = get_store()
    namespace = namespace_for(user_id)
    items = store.search(namespace, query="all facts")
    for item in items:
        store.delete(namespace, item.key)
    return len(items)
```

Your turn: wire the `forget_me` command into `main.py` 's loop, before the normal `graph.invoke(...)` call.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the honest fresh-recall test yourself, using this skeleton:

```
# test_memory_keeper.py (adds to Steps 1-3's tests)
from store import get_store, namespace_for, forget_user

def test_forget_user_actually_emptyies_the_namespace():
    store = get_store()
    namespace = namespace_for("alex")
    store.put(namespace, "preferred_name", {"text": "Alex"})
    store.put(namespace, "stated_preference", {"text": "Likes short answers"})

    deleted_count = forget_user("alex")
    assert deleted_count == 2

    remaining = store.search(namespace, query="anything")
    assert remaining == []

def test_fresh_thread_after_forgetting_has_no_recall():
    # your turn: run the same seed -> forget -> ask-on-a-brand-new-thread sequence
    # this test used in Step 2 (test_new_thread_recalls_a_seeded_fact), but call
    # forget_user in between the seed and the ask, and assert the OPPOSITE outcome
    # the answer should now show no knowledge of the user at all
    ...

if __name__ == "__main__":
    test_forget_user_actually_emptyies_the_namespace()
    print("PASS: forget_user removes every key in the user's namespace")
    test_fresh_thread_after_forgetting_has_no_recall()
    print("PASS: a brand-new thread, after forgetting, genuinely knows nothing")
```

Notice the second test is deliberately the mirror image of Step 2's

`test_new_thread_recalls_a_seeded_fact` — same shape, opposite expected outcome, because `forget_user` ran in between. Writing it this way makes it obvious, just from reading the two tests side by side, that this step didn't just add a delete function — it proved the delete actually undoes what Step 2 and Step 3 built.

Compare your finished `forget_user`, `main.py` command handling, and test against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic sketches the search-then-delete loop and where to intercept the command. Intermediate gives the real, nearly-complete `forget_user` function. Advanced gives the honest test shape — proving the namespace is actually empty after forgetting, and proving a brand-new thread genuinely can't recall anything afterward, mirroring the exact test Step 2 used to prove recall in the first place.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)